

Toollery: Scaling LLM Agents to Hundreds of Tools

Anonymous ACL submission

Abstract

The growing deployment of large language models (LLMs) as agentic systems has made tool selection a critical bottleneck in real-world applications. We identify a *tool pool scalability challenge*, where tool selection accuracy degrades significantly as the number of available tools increases, even for tool-specialized models. To quantify this effect, we introduce **ScaleTool**, a benchmark framework for the controlled expansion of candidate tool pools. Using ScaleTool, we observe that scaling the candidate set to 1,000 tools incurs a 35-fold increase in input tokens, frequently triggering context length limits. Even when within these limits, models suffer from a significant decline in selection accuracy. In either cases, standard LLM-based tool selection becomes infeasible. To address this, we propose **Toollery**, a framework for robust tool selection under growing tool pools without additional LLM fine-tuning. Toollery bridges the semantic gap between user intent and tool functionality through a two-stage design: offline synthesis and self-verification of proxy queries using teacher and verifier LLMs, followed by on-line intent-intent retrieval constructing a compact, high-confidence candidate set. Experiments demonstrate that Toollery maintains a nearly constant candidate set size as the tool pool grows, while achieving an improvement at least 10% at challenged scenarios for baseline LLMs.

1 Introduction

The rapid evolution of large language models (LLMs) has endowed them with strong capabilities in natural language understanding, reasoning, and program synthesis (Achiam et al., 2023). Beyond their role as standalone text generators, LLMs are increasingly deployed as agentic systems that interact with external environments through tool invocation (Huang et al., 2024;

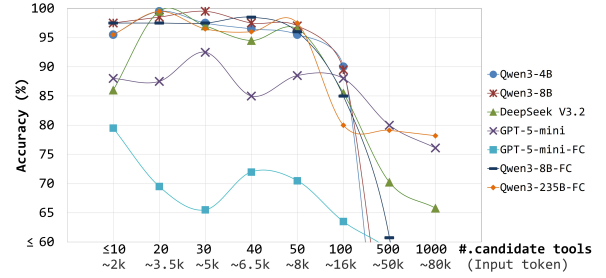


Figure 1: Impact of candidate pool size on tool selection accuracy. The graph illustrates the performance degradation across various LLMs as the number of available tools (n) increases from 10 to 1000. Results are derived from a synthetic benchmark evaluating queries from BFCL-V4.

Schick et al., 2023; Yao et al., 2022). A central component of this paradigm is *tool selection*—the process by which an agent identifies the appropriate tool(s) to invoke from a set of candidates.¹

Despite recent progress, deploying tool-augmented LLM agents remains challenging as candidate tool sets grow and user requests remain underspecified. We identify a pronounced *tool pool scalability challenge*: tool selection accuracy degrades as the number of available tools increases. To study this effect in a controlled setting, we evaluate LLMs on verified (query, tool) pairs drawn from HammerBench (Wang et al., 2025) and xLAM (Liu et al., 2024b), systematically expanding the candidate tool set from 10 to 1,000 by injecting randomly sampled distractor tools while holding the ground-truth tool fixed. In this setting, accuracy drops consistently as the candidate set expands (1). Similar degradation is observed in broader evaluations, such as the

¹In this line of research, the terms *tool*, *API*, and *function* are used interchangeably. For consistency, we use the term *tool* throughout this paper.

Berkeley Function Calling Leaderboard (BFCL) (Patil et al., 2025), where larger and more dynamic tool environments (the *live-multiple* dataset from BFCL) lead to reduced accuracy.

In practical deployments, this challenge is amplified. Real-world agents often operate over massive tool libraries, ranging from dozens in OS automation (Wu et al., 2024) to over 10,000 operations in cloud infrastructure SDKs (Amazon Web Services, 2024). Recent benchmarks like LiveMCPBench (Mo et al., 2025) and TPS-Bench (Xu et al., 2025) confirm this trend, validating agents against ecosystems containing hundreds of diverse tools. At this scale, the standard approach—attaching all tool definitions to the system prompt—is impractical for two reasons. First, computational and token costs scale prohibitively with context length. Second, as our empirical results demonstrate, expanding the tool context substantially reduces selection accuracy. These observations motivate the need for mechanisms that explicitly manage and compress the tool search space prior to model inference.

To address this challenge, we propose **Toollery**, a modular framework designed to enable accurate tool selection over growing tool pools without additional model fine-tuning. Unlike prior retrieval-based approaches that match user queries directly to tool metadata, Toollery introduces a self-verified proxy-query layer that enables *intent-intent matching*, decoupling semantic grounding from model inference. Toollery operates by (i) pre-generating structured, self-verified documentation for each tool, and (ii) dynamically filtering the tool space at inference time to present the LLM with a compact set of high-confidence candidates. This design allows existing LLMs to retain their function-calling capabilities while adapting to a continuously growing tool library.

Our contributions are as follows:

- **The Toollery Framework.** We introduce a modular framework that operates purely at the tool-selection interface. It integrates with diverse backbone LLMs without modifying model weights, enabling robust tool selection under growing tool pools.
- **Self-verified Tool Manuals.** We propose an automated pipeline that synthesizes structured tool manuals composed of diverse natural-language proxy queries. These

are rigorously filtered via round-trip self-verification, improving the semantic alignment between user intent and tool functionality.

- **ScaleTool Benchmark.** We construct ScaleTool, a benchmark derived from HammerBench (Wang et al., 2025) and xLAM (Liu et al., 2024b) that evaluates tool selection robustness under controlled candidate-set growth. Experiments across multiple benchmarks demonstrate consistent improvements as the candidate tool set expands.

Taken together, these observations motivate a solution that explicitly manages candidate-set growth without relying on additional model training or architectural modification. In the following 3, we introduce Toollery in detail, a modular framework that addresses the tool pool scalability challenge by separating semantic grounding from inference-time decision making. We first describe how Toollery constructs self-verified tool manuals offline, and then explain how these manuals enable efficient intent-intent matching to dynamically filter candidate tools at inference time.

2 Related Work

Recent work on integrating external tools into LLMs broadly follows two directions: (i) internalizing tool-use capabilities through model retraining or fine-tuning, and (ii) mitigating scalability challenges through retrieval and strategy optimization.

Model retraining and fine-tuning. A primary category of research aims to internalize tool-use behavior via supervised fine-tuning (SFT). Seminal works like Gorilla (Patil et al., 2024; Touvron et al., 2023) and ToolLLM (Qin et al., 2023) demonstrate that training on tools or execution trajectories substantially improves syntactic correctness and reduces hallucinations compared to prompting alone. Subsequent efforts have specialized this paradigm for conversational alignment (Liu et al., 2024a; Acikgoz et al., 2025), on-device efficiency (Erdogan et al., 2024), and robustness to noisy descriptions (Wang et al., 2025). Recently, proprietary models like GLM-4 (Zeng et al., 2025) have further integrated native function-calling directly into pre-training and alignment.

While effective for specific tasks, internalizing tool-use capabilities via SFT introduces a

rigid trade-off. Fine-tuning an LLM on specialized tool-invocation datasets often degrades the model’s broader reasoning and general-purpose capabilities—a phenomenon characterized as “catastrophic forgetting” or alignment tax (Kirkpatrick et al., 2017; Luo et al., 2025; Fu et al., 2024). In contrast, Toollery offers a non-destructive, plug-and-play alternative. By decoupling tool-selection logic from the model’s internal weights, we enable robust selection across growing tool pools without sacrificing the backbone LLM’s inherent reasoning versatility.

Retrieval and strategy optimization. Apart from fine-tuning, another category of work treats tool selection as a retrieval problem, introducing external mechanisms to manage both context length and the combinatorial growth of the action space. ToolRM (Agarwal et al., 2025) proposes a learned dense retriever to rank candidate tools by semantic relevance, decoupling tool selection from execution. MassTool (Lin et al., 2025) extends this idea to extreme-scale settings, leveraging indexing and search techniques to operate over libraries containing tens of thousands of tools.

Complementary approaches optimize how tool information is presented to the LLM. The Less is More (Paramanayakam et al., 2025) demonstrates that aggressive context pruning—rather than maximal context inclusion—often improves tool selection accuracy. Collectively, these methods aim to construct a compact, high-confidence candidate set that bridges the gap between massive tool libraries and the limited attention budget of LLMs.

Unlike prior retrieval-based approaches that rely on direct query–metadata matching or learned retrievers, Toollery introduces a self-verified proxy-query layer. This explicitly aligns user intent with tool functionality, decoupling semantic grounding from model inference and eliminating the need for retriever training.

3 The Toollery Design

The essential idea of Toollery is to let LLMs study each single tool beforehand, so that at inference time the selection challenge is reduced to an easier comparison problem. By isolating the tool selection problem into online and offline stages, Toollery can be readily plugged in any existing LLMs and play. As illustrated in 2, these two stages do the following:

- **Offline Tool Manual Synthesis:** A pre-

processing phase that bridges the semantic gap between static tool metadata and diverse user intents.

- **Online Proxy-Query Matching:** An inference-time mechanism that dynamically filters the search space to retrieve a compact, high-confidence candidate tool set.

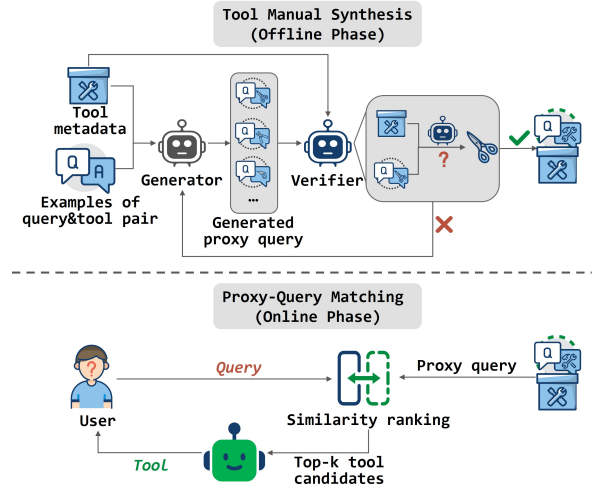


Figure 2: Overview of the Toollery framework. The pipeline bridges the semantic gap in tool selection via two stages: (1) **Tool Manual Synthesis** (offline), which generates and validates proxy queries, and (2) **Proxy-Query Matching** (online), which retrieves relevant tools for the LLM.

3.1 Tool Manual Synthesis (Offline Phase)

A core limitation of conventional retrieval-based tool selection lies in the semantic gap between abstract tool metadata and the diverse, often ambiguous expressions of user intent. While tool metadata specifies *functionality* (what a tool does), users typically describe *intent* (what they want). A natural approach to bridging this gap is to synthesize natural-language queries from tool descriptions to serve as semantic anchors. However, naïvely generated synthetic queries are often noisy or weakly aligned: a query may be linguistically plausible yet compatible with multiple tools, or it may map to an incorrect tool when evaluated against a larger candidate set. Such ambiguity is particularly harmful under *candidate-set growth*, where even small semantic drifts can substantially degrade retrieval precision. To address this, Toollery constructs a **Tool User Manual** that encodes high-precision intent–tool associations. This is achieved via a two-step pro-

cess: diversity-driven synthesis followed by rigorous self-verification (see 1).

Diversity-Driven Query Synthesis. For each tool, we treat its metadata as a semantic seed to generate a diverse population of proxy user queries. Using a high-capacity teacher LLM (e.g., GPT-5), we apply chain-of-thought prompting to iteratively transform tool metadata into natural language intents. We explicitly steer the generation using three linguistic operators:

- **Paraphrasing:** Rewriting the intent using varied vocabulary and syntactic structures.
- **Complexity Injection:** Incorporating implicit constraints or multi-turn context (e.g., transforming "Get weather" to "I need to know if I should pack an umbrella for London tomorrow").
- **Ambiguity Simulation:** Simulating underspecified user requests that rely on partial information, forcing the model to infer the correct tool from context.

This approach ensures the resulting manual covers a broad spectrum of potential user interactions, extending far beyond the semantic reach of static documentation.

Self-Verification via Round-Trip Consistency. Automated synthesis risks introducing "hallucinated" queries that do not align with the tool’s actual functionality. To mitigate this, we deploy a *Verifier Agent*—a separate LLM instance for quality control. The Verifier is presented with a generated proxy query q along with a candidate set containing the target tool t and a set of distractor tools sampled from the pool. We then perform a **Round-Trip Consistency Check**: if the Verifier retrieves the original seed tool t given query q , the query is deemed valid and retained. If the agent selects a different tool or fails to select any tool, the query is discarded. This peer-review process ensures that every entry in the Tool User Manual is not only linguistically diverse but also functionally unambiguous.

3.2 Proxy-Query Matching (Online Phase)

As illustrated in 3, providing an LLM with a massive set of candidate tools at inference time is both inefficient and error-prone. Rather than presenting the full library, Toolery leverages the pre-constructed **Tool User Manual** to selectively filter the search space, enabling the model to reason

Algorithm 1 Tool Manual Synthesis with Round-Trip Verification

Input: Tool pool $\mathcal{T}$; metadata $M(t)$ for each $t \in \mathcal{T}$; Teacher $\mathcal{L}_{\text{gen}}$; Verifier $\mathcal{L}_{\text{ver}}$; distractor count k

Output: Tool User Manual $\mathcal{U}$

Initialize $\mathcal{U} \leftarrow \emptyset$

for each tool $t \in \mathcal{T}$ **do**

▷ Step 1: Diversity-Driven Synthesis

Generate candidate queries $Q_t = \{q_1, \dots, q_N\}$ from $M(t)$ using $\mathcal{L}_{\text{gen}}$

for each query $q \in Q_t$ **do**

▷ Step 2: Round-Trip Verification

Sample distractor set $\mathcal{S}_{\text{neg}} \subset \mathcal{T} \setminus \{t\}$ with $|\mathcal{S}_{\text{neg}}| = k$

Construct verification pool $\mathcal{T}_{\text{verify}} = \{t\} \cup \mathcal{S}_{\text{neg}}$

$\hat{t} \leftarrow \mathcal{L}_{\text{ver}}(\text{query} = q, \text{tools} = \mathcal{T}_{\text{verify}})$

if $\hat{t} = t$ **then**

Add (q, t) to $\mathcal{U}$

end if

end for

end for

return $\mathcal{U}$

over a compact, semantically coherent candidate set.

Intent-Level Semantic Retrieval. At inference time, Toolery avoids direct retrieval against abstract tool metadata. Instead, it embeds the incoming user query into a shared semantic space and performs retrieval against the verified proxy queries stored in the Tool User Manual. In our implementation, we employ a lightweight embedding model (e.g., Google Gemma 300M or similar) to encode both user and proxy queries into dense vector representations. Retrieval is performed by computing the cosine similarity between the user query embedding and the proxy-query index. Because each proxy query is self-verified to map unambiguously to a specific tool, this process operates over a space of grounded user intents rather than static descriptions. This reformulation fundamentally transforms tool selection from *Query-to-Metadata* matching into *Query-to-Query* alignment—a paradigm that is significantly more robust to linguistic variation, particularly as the tool pool scales.

Dynamic Candidate Set Construction. The retrieved proxy queries are aggregated to form a ranked list of *candidate tools*. Since multiple

Table 1: ScaleTool-HammerBench test results

Model	≤ 10	20	30	40	50	100	500	1000
Qwen3-4B (Prompt)	49.67	55.60	52.20	45.00	55.20	50.30	31.25	\
Qwen3-4B (Prompt) + Toollery	72.67 $\uparrow$	64.20 $\uparrow$	64.60 $\uparrow$	69.60 $\uparrow$	68.40 $\uparrow$	64.15 $\uparrow$	67.50 $\uparrow$	66.50 $\uparrow$
Qwen3-8B (Prompt)	72.33	76.20	73.40	72.20	75.40	60.20	56.25	\
Qwen3-8B (Prompt) + Toollery	72.00	76.40 $\uparrow$	71.40	82.20 $\uparrow$	80.40 $\uparrow$	75.50 $\uparrow$	62.50 $\uparrow$	62.00 $\uparrow$
DeepSeek V3.2 (Prompt)	97.00	77.20	75.20	81.80	77.40	75.00	70.00	63.00
DeepSeek V3.2 (Prompt) + Toollery	96.67	86.60 $\uparrow$	88.20 $\uparrow$	89.80 $\uparrow$	82.40 $\uparrow$	89.85 $\uparrow$	85.75 $\uparrow$	78.00 $\uparrow$
GPT-5-mini (Prompt)	89.00	84.20	85.80	83.60	90.40	70.00	62.50	54.00
GPT-5-mini (Prompt) + Toollery	89.67 $\uparrow$	85.00 $\uparrow$	87.80 $\uparrow$	84.40 $\uparrow$	95.00 $\uparrow$	85.50 $\uparrow$	78.00 $\uparrow$	78.00 $\uparrow$
GPT-5-mini (FC)	40.33	44.60	27.80	31.20	22.40	10.00	2.50	\
GPT-5-mini (FC) + Toollery	44.67 $\uparrow$	47.20 $\uparrow$	44.60 $\uparrow$	51.20 $\uparrow$	44.40 $\uparrow$	45.50 $\uparrow$	42.95 $\uparrow$	44.40 $\uparrow$
Qwen3-8B (FC)	69.67	74.20	73.60	68.60	67.00	61.50	30.50	\
Qwen3-8B (FC) + Toollery	91.33 $\uparrow$	86.40 $\uparrow$	89.00 $\uparrow$	84.80 $\uparrow$	82.20 $\uparrow$	85.50 $\uparrow$	75.80 $\uparrow$	77.50 $\uparrow$
Qwen3-235B (FC)	73.67	73.40	64.60	63.00	64.80	35.00	11.25	\
Qwen3-235B (FC) + Toollery	86.33 $\uparrow$	85.40 $\uparrow$	84.60 $\uparrow$	82.20 $\uparrow$	76.40 $\uparrow$	82.50 $\uparrow$	75.00 $\uparrow$	70.50 $\uparrow$

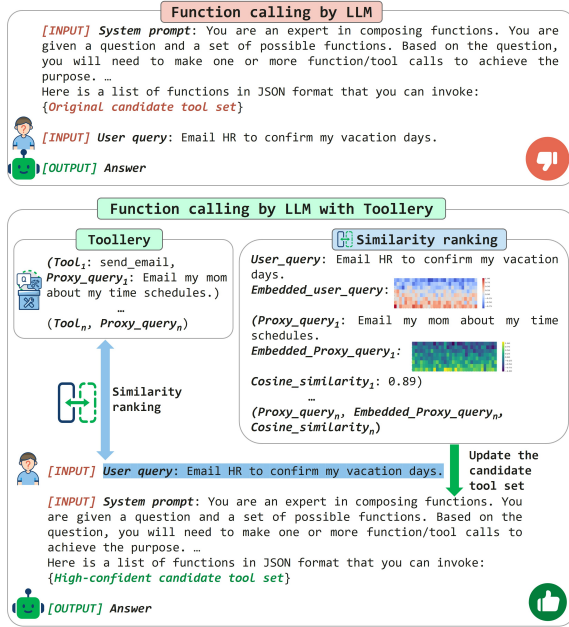


Figure 3: Comparison of tool selection pipelines. Standard methods (upper) pass all candidates to the LLM, leading to context overflow. Toollery (lower) filters candidates via proxy-query matching before inference.

proxy queries may map to the same underlying tool, we aggregate their similarity scores to derive a tool-level relevance score. This aggregation step acts as a denoising filter: it suppresses isolated false matches while reinforcing tools supported by multiple high-confidence proxy hits. From this ranked list, Toollery selects the top- k tools and instantiates their full specifications (names, descriptions, and parameter schemas). By restricting the context to this compact subset, Toollery minimizes contextual noise, allowing the model to focus on

precise argument extraction.

Inference and Execution. Finally, the compact candidate set is passed to the inference LLM using standard function-calling protocols. The LLM acts as the final decision-maker, selecting the appropriate tool and generating the arguments based on the user query. Importantly, Toollery does not override the LLM’s reasoning capabilities; instead, it constrains the *action space*, ensuring the model operates within a focused and semantically relevant boundary. This design preserves compatibility with existing agentic pipelines while ensuring robustness against candidate-set growth.

4 Experiments

To rigorously quantify the tool pool scalability challenge and evaluate the efficacy of Toollery, we introduce **ScaleTool**, a benchmark framework that systematically varies context density while preserving ground-truth intent.

4.1 The ScaleTool Protocol

Existing benchmarks typically evaluate tool selection with a fixed, often small, number of candidate tools. ScaleTool generalizes this by treating the *candidate set size* as an independent variable. For a given verified pair (query, t_{gt}) where t_{gt} is the ground-truth tool, we construct a candidate pool $\mathcal{P}_n$ of size $n \in \{2, 5, 10, \dots, 500, 1000\}$. The pool $\mathcal{P}_n$ contains t_{gt} and $n - 1$ distractor tools sampled uniformly from the disjoint set of available tools in the library. This construction isolates the impact of contextual noise: as n increases, the model must identify t_{gt} in the context of an increasing volume of irrelevant tool definitions.

Crucially, this expansion causes a dramatic in-

crease in input complexity. As shown in our analysis, expanding the candidate set from 10 to 1,000 tools results in input lengths growing from $\sim 1\text{k}$ to over 100k tokens. This extreme context loading allows us to stress-test the capability of LLMs and demonstrate the necessity of pre-inference filtering.

4.2 Evaluation Datasets

We apply the ScaleTool protocol to three widely used tool-use datasets, creating three scaled benchmarks.

ScaleTool-HB (HammerBench). Derived from HammerBench (Wang et al., 2025), this subset comprises 6,531 queries spanning 60 functional categories and a total library of 1,063 tools. It evaluates robustness against noisy or imperfect tool descriptions.

ScaleTool-xLAM. Adapted from xLAM (Liu et al., 2024b), this benchmark includes 60,000 high-quality function-calling instances covering 3,673 executable tools across 21 categories. It represents a high-scale scenario with a diverse range of user intents.

ScaleTool-BFCL. We also adapt the Berkeley Function Calling Leaderboard (BFCL-V4) (Patil et al., 2025), the industry standard for executable correctness. Unlike surface-form matching, BFCL utilizes Abstract Syntax Tree (AST) matching to verify the syntactic validity of generated calls. We evaluate on both the **Non-live** and **Live** multiple-tool subsets. Under ScaleTool expansion, the Live subset reaches context lengths exceeding 100k tokens, providing our most rigorous test of long-context robustness.

4.3 Baselines and Experimental Setup

We evaluate Toolllery across a diverse spectrum of foundation models to ensure our findings generalize across architectures and capabilities.

Foundation Models. We select representative models from both open-source and closed-source families, including the Qwen3 series (Yang et al., 2025), DeepSeek (Liu et al., 2025), and GPT-5-mini (OpenAI). These models were chosen to cover a wide range of parameter counts and reasoning capabilities.

Interaction Interfaces. To verify that Toolllery is independent of the tool-use implementation, we evaluate two distinct interaction paradigms:

- **Native Function Calling (FC):** The model

utilizes specific architectural tokens or structured decoding (e.g., JSON mode) explicitly optimized for tool use.

- **Prompt-Based Tool Calling:** The model receives tool definitions as natural language text within the system prompt and is instructed to generate tool calls as formatted text, without special architectural support.

This dual-interface setup allows us to assess whether Toolllery effectively supports both specialized agents and general-purpose LLMs.

4.4 Results and Analysis

While overall benchmark scores provide a high-level view of model capability, they obscure the critical impact of scalability. To isolate the effect of tool library size, we utilize the ScaleTool protocol, holding the query–tool ground truth fixed while varying the candidate pool size N . This controlled setting allows us to characterize the performance decay induced by distractors and evaluate Toolllery’s ability to improve inference accuracy across different backbone LLMs. We report tool selection accuracy binned by candidate set size for all three benchmarks.

4.4.1 Results on ScaleTool-HB

As detailed in 1, candidate-set growth induces a severe performance penalty on backbone LLMs, often leading to total functional collapse in the large-context setting ($N \geq 500$). Contrary to the expectation that specialized "function-calling" (FC) models are more robust, we observe that they are exceptionally brittle to candidate-set growth. High-capacity models like Qwen3-235B (FC) and GPT-5-mini (FC) struggle significantly as the tool pool expands. For instance, Qwen3-235B (FC) maintains respectable accuracy at small scales but collapses to 11.25% at $N = 500$. Similarly, GPT-5-mini (FC) fails almost entirely, dropping to 2.5% accuracy. This suggests that native FC mechanisms rely on precise attention to schema definitions, which is easily disrupted by the "noise" of distractor tools. Toolllery effectively prevents the degradation for these models. For Qwen3-235B (FC), Toolllery restores accuracy to 70.50% at $N = 1000$ —a tool size scale where the baseline failed to produce valid outputs. This indicates that pre-filtering is not just an optimization but a necessity for making native FC viable at scale.

Table 2: ScaleTool-xLAM test results

Model	≤ 10	20	30	40	50	100	500	1000
Qwen3-4B (Prompt)	86.33	92.00	84.40	87.20	82.00	61.59	39.88	\
Qwen3-4B (Prompt) + Toollery	88.00 $\uparrow$	92.00	88.80 $\uparrow$	91.40 $\uparrow$	91.40 $\uparrow$	90.71 $\uparrow$	81.25 $\uparrow$	72.00 $\uparrow$
Qwen3-8B (Prompt)	90.33	83.00	87.20	88.60	80.40	66.31	41.25	\
Qwen3-8B (Prompt) + Toollery	88.33	89.80 $\uparrow$	91.20 $\uparrow$	88.20	90.00 $\uparrow$	90.00 $\uparrow$	87.50 $\uparrow$	78.95 $\uparrow$
DeepSeek V3.2 (Prompt)	88.67	93.60	95.00	92.60	89.20	85.00	83.75	16.56
DeepSeek V3.2 (Prompt) + Toollery	90.67 $\uparrow$	94.40 $\uparrow$	93.20	92.80 $\uparrow$	96.00 $\uparrow$	90.00 $\uparrow$	86.25 $\uparrow$	79.47 $\uparrow$
GPT-5-mini (Prompt)	88.67	92.00	92.80	92.00	89.40	84.13	80.50	71.59
GPT-5-mini (Prompt) + Toollery	91.67 $\uparrow$	91.40	95.40 $\uparrow$	91.40	90.20 $\uparrow$	89.56 $\uparrow$	81.50 $\uparrow$	79.64 $\uparrow$
GPT-5-mini (FC)	91.00	89.40	94.00	89.80	87.20	70.00	42.50	11.37
GPT-5-mini (FC) + Toollery	91.33 $\uparrow$	91.80 $\uparrow$	93.20	91.60 $\uparrow$	88.80 $\uparrow$	90.51 $\uparrow$	73.75 $\uparrow$	69.86 $\uparrow$
Qwen3-8B (FC)	89.33	86.20	90.60	92.40	86.00	68.28	47.50	\
Qwen3-8B (FC) + Toollery	90.67 $\uparrow$	90.80 $\uparrow$	93.60 $\uparrow$	90.20	90.40 $\uparrow$	91.06 $\uparrow$	81.25 $\uparrow$	73.54 $\uparrow$
Qwen3-235B (FC)	80.00	91.40	93.80	92.40	89.40	87.16	76.25	33.21
Qwen3-235B (FC) + Toollery	90.00 $\uparrow$	94.80 $\uparrow$	91.60	92.60 $\uparrow$	92.20 $\uparrow$	90.83 $\uparrow$	81.25 $\uparrow$	77.19 $\uparrow$

Table 3: BFCL-V4 non-live multiple results

Model	≤ 10	20	30	40	50	100	500	1000
Qwen3-4B (Prompt)	95.50	99.50	97.50	96.50	95.50	90.00	24.25	\
Qwen3-4B (Prompt) + Toollery	100.00 $\uparrow$	97.50	98.00 $\uparrow$	95.00	98.00 $\uparrow$	91.50 $\uparrow$	83.63 $\uparrow$	81.50 $\uparrow$
Qwen3-8B (Prompt)	97.50	98.50	99.50	97.50	97.00	89.50	37.63	\
Qwen3-8B (Prompt) + Toollery	99.50 $\uparrow$	100.00 $\uparrow$	97.50	96.00	98.00 $\uparrow$	91.00 $\uparrow$	80.63 $\uparrow$	80.40 $\uparrow$
DeepSeek V3.2 (Prompt)	86.00	99.50	97.00	94.50	96.50	85.50	70.25	65.80
DeepSeek V3.2 (Prompt) + Toollery	97.50 $\uparrow$	97.50	95.00	98.00 $\uparrow$	98.50 $\uparrow$	89.50 $\uparrow$	81.38 $\uparrow$	80.80 $\uparrow$
GPT-5-mini (Prompt)	88.00	87.50	92.50	85.00	88.50	88.00	80.00	76.10
GPT-5-mini (Prompt) + Toollery	88.00	89.50 $\uparrow$	91.50	86.50 $\uparrow$	95.00 $\uparrow$	90.00 $\uparrow$	82.00 $\uparrow$	81.00 $\uparrow$
GPT-5-mini (FC)	79.50	69.50	65.50	72.00	70.50	63.50	58.63	50.50
GPT-5-mini (FC) + Toollery	81.50 $\uparrow$	77.00 $\uparrow$	75.50 $\uparrow$	73.00 $\uparrow$	76.00 $\uparrow$	69.50 $\uparrow$	68.63 $\uparrow$	68.50 $\uparrow$
Qwen3-8B (FC)	97.50	97.50	97.50	98.50	96.00	85.00	60.75	\
Qwen3-8B (FC) + Toollery	99.50 $\uparrow$	96.50	97.50	96.50	97.00 $\uparrow$	90.50 $\uparrow$	86.75 $\uparrow$	83.30 $\uparrow$
Qwen3-235B (FC)	95.50	99.50	96.50	96.00	97.50	80.00	79.13	78.20
Qwen3-235B (FC) + Toollery	96.00 $\uparrow$	99.00	98.50 $\uparrow$	98.00 $\uparrow$	98.00 $\uparrow$	93.50 $\uparrow$	94.00 $\uparrow$	86.30 $\uparrow$

A noticeable finding is the impact on smaller architectures. The baseline Qwen3-4B (Prompt) struggles significantly, achieving only 31.25% at $N = 500$ and failing completely at $N = 1000$. With Toollery, this model achieves 66.50% accuracy at $N = 1000$, effectively matching the performance of the much larger baseline Qwen3-8B. This result contradicts the assumption that small models lack the reasoning capacity to benefit from retrieval; rather, it suggests that their limitation is *context processing*, which Toollery resolves by presenting a compact, high-relevance window. For robust prompt-based models like DeepSeek V3.2 and GPT-5-mini, Toollery provides a consistent boost. While these models have strong baselines ($> 60\%$ at $N = 1000$), Toollery pushes them further, achieving state-of-the-art performance (e.g., DeepSeek V3.2 + Toollery reaches 78.00% at $N = 1000$ vs. 63.00% baseline). In summary, Toollery acts as an anchor for strong models and an enabler for weaker or brittle ones.

4.4.2 Results on ScaleTool-xLAM

As shown in 2, the ScaleTool-xLAM benchmark presents a "high-stakes" environment: while models start with high baseline accuracy in small contexts ($N \leq 50$), they experience sharply declines as the candidate pool expands, often collapsing more severely than in HammerBench.

A critical observation is that despite high initial performance, many baselines effectively cease to function in the large-context setting ($N = 1000$). Both Qwen3-4B and Qwen3-8B (Prompt and FC) degrade rapidly beyond $N = 100$. At $N = 1000$, these baselines fail completely, due to context window exhaustion. Even powerful models like DeepSeek V3.2 and GPT-5-mini (FC) hit a "performance cliff." For instance, DeepSeek V3.2 maintains $\sim 85\%$ accuracy at $N = 100$ but drops down to 16.56% at $N = 1000$. Similarly, GPT-5-mini (FC) drops from 87.20% ($N = 50$) to 11.37% ($N = 1000$). Toollery reverses this collapse. It restores the failed Qwen3 models to functional util-

ity ($> 70\%$ accuracy at $N = 1000$) and boosts DeepSeek V3.2 from 16.56% to 79.47%. This demonstrates that the "collapse" is not a reasoning failure, but a search failure—one that Toollery effectively solves.

Unlike the HammerBench results where gains were sometimes model-dependent, Toollery provides a near-universal benefit on xLAM. At $N = 1000$, Toollery brings a diverse array of models—ranging from the compact Qwen3-4B (72.00%) to the massive Qwen3-235B (77.19%)—to a remarkably similar high-performance band. This suggests that with effective candidate filtering, smaller models can punch significantly above their weight class, matching the performance of much larger baselines. Even for the strongest baseline, GPT-5-mini (Prompt), which degrades less severe (71.59% at $N = 1000$), Toollery still yields a significant margin of improvement ($\sim \uparrow 8\%$ to 79.64%). This confirms that even state-of-the-art long-context models benefit from decoupling the retrieval space from the reasoning space.

4.4.3 Results on ScaleTool-BFCL

The results on ScaleTool-BFCL (3 and 4) validate our findings in the most rigorous, execution-based setting available. Unlike prior benchmarks, BFCL enforces strict syntactic correctness via Abstract Syntax Tree (AST) matching. A success here implies that the model correctly comprehended the tool’s schema and generated valid arguments.

In the Non-Live dataset (3), we observe that providing too many tool definitions overwhelms the model’s ability to attend to specific parameter schemas. Smaller models like Qwen3-4B and Qwen3-8B (both Prompt and FC) exhibit near-perfect performance at small scales ($> 95\%$) but face significant failure at $N = 500$ (dropping to 24 – 37%) and total collapse at $N = 1000$. This indicates that while these models can reason correctly, their context windows are easily saturated by irrelevant schema tokens. Toollery effectively aids these models. At $N = 1000$, it boosts Qwen3-4B from failure to 81.50% and Qwen3-8B (FC) from failure to 83.30%. By presenting only the top- k relevant schemas, Toollery allows these efficient models to dedicate their attention to argument generation rather than filtering noise. Even the massive Qwen3-235B (FC), which is relatively robust (78.20% baseline at $N = 1000$), benefits significantly, jumping to 86.30% with Toollery. This confirms that even the largest models suffer

when performing complex tool selection.

The Live dataset (4) represents the most challenging scenario, involving real-world tools with often overlapping or ambiguous functionality. The vulnerability of native FC interfaces is more obvious here. Qwen3-8B (FC), which performs well at small scales (93.50%), collapses precipitously to 15.90% at $N = 1000$, while Toollery restores its accuracy to 83.50%. Across all models, Toollery flattens the performance curve. For example, DeepSeek V3.2 sees its accuracy at $N = 1000$ rise from 73.80% to 85.50%. The consistency of these gains implies that Toollery’s "intent–intent" matching is highly effective at disentangling the semantic overlaps often found in real-world tool libraries.

Our evaluation across three benchmarks reveals two fundamental insights:

- **Performance Degradation Across LLMs.** Candidate-set growth degrades performance across all interfaces. Notably, Native FC models exhibit extreme sensitivity, often suffering steeper drops (e.g., to $< 15\%$ accuracy on BFCL-Live) than robust prompt-based baselines. Conversely, prompt-based models—especially smaller architectures—suffer primarily from context overload.
- **High-Performance Tool Use with Toollery.** Toollery improves accuracy under growth compared to baseline LLMs. On xLAM and BFCL, small models with Toollery (e.g., Qwen3-4B) frequently outperform larger baselines without it (e.g., Qwen3-8B). This demonstrates that the bottleneck for small LLM is *context processing*, and Toollery enables it to operate reliably in massive tool environments.

While Toollery focuses on inference-time mitigation, we also explore supervised fine-tuning as an alternative. As detailed in Appendix ??, fine-tuning on ScaleTool-generated data is effective but comes with significant training and deployment overheads, whereas Toollery offers a lightweight, plug-and-play solution.

5 Discussion and Future Work

As LLMs transition from passive text generators to active agents, their ability to select the correct tools from massive, evolving libraries becomes a fundamental bottleneck. This paper identifies

Table 4: BFCL-V4 live multiple results

Model	≤ 10	20	30	40	50	100	500	1000
Qwen3-4B (Prompt)	85.50	86.00	88.00	82.50	78.00	69.50	24.88	\
Qwen3-4B (Prompt) + Toolllery	87.00 $\uparrow$	84.50	87.00	86.50 $\uparrow$	82.50 $\uparrow$	81.00 $\uparrow$	83.75 $\uparrow$	81.20 $\uparrow$
Qwen3-8B (Prompt)	97.00	93.50	92.00	84.00	83.00	84.50	38.50	\
Qwen3-8B (Prompt) + Toolllery	95.50	98.50 $\uparrow$	98.50 $\uparrow$	92.50 $\uparrow$	94.50	86.50 $\uparrow$	79.38 $\uparrow$	80.30 $\uparrow$
DeepSeek V3.2 (Prompt)	94.50	80.00	84.00	89.50	79.50	63.00	77.00	73.80
DeepSeek V3.2 (Prompt) + Toolllery	98.00 $\uparrow$	78.50	94.50 $\uparrow$	99.50 $\uparrow$	89.50 $\uparrow$	89.00 $\uparrow$	83.13 $\uparrow$	85.50 $\uparrow$
GPT-5-mini (Prompt)	83.00	93.00	90.00	87.00	88.00	88.00	79.75	75.20
GPT-5-mini (Prompt) + Toolllery	83.50 $\uparrow$	94.00 $\uparrow$	89.50	95.00 $\uparrow$	89.50 $\uparrow$	89.00 $\uparrow$	84.50 $\uparrow$	78.80 $\uparrow$
GPT-5-mini (FC)	67.00	62.00	63.50	62.00	61.00	63.50	57.00	50.50
GPT-5-mini (FC) + Toolllery	74.50 $\uparrow$	67.50 $\uparrow$	69.00 $\uparrow$	69.50 $\uparrow$	67.00 $\uparrow$	67.50 $\uparrow$	66.00 $\uparrow$	65.30 $\uparrow$
Qwen3-8B (FC)	93.50	94.50	92.50	83.00	83.50	83.00	60.00	15.90
Qwen3-8B (FC) + Toolllery	93.50	98.00 $\uparrow$	90.50	99.00 $\uparrow$	91.50 $\uparrow$	88.50 $\uparrow$	85.88 $\uparrow$	83.50 $\uparrow$
Qwen3-235B (FC)	98.00	93.00	87.50	89.50	85.50	80.00	81.13	80.10
Qwen3-235B (FC) + Toolllery	98.50 $\uparrow$	92.50	89.00 $\uparrow$	89.50	90.00	91.50 $\uparrow$	88.63 $\uparrow$	87.30 $\uparrow$

and quantifies the *tool pool scalability challenge*, demonstrating that candidate-set growth imposes severe pressure on LLM-based agents. ScaleTool shows that this degradation is not merely a capacity issue but a structural limitation of attending to large, noisy search spaces.

To address this, we introduced **Toolllery**, a training-free framework that aids LLM on tool selection via intent–intent matching. Our extensive evaluation confirms that Toolllery consistently improves performance across diverse LLM architectures and interfaces. It proves effective in large tool candidate set, enabling even smaller LLMs to compete with larger LLM baselines. While supervised fine-tuning can also mitigate these issues (Appendix ??), Toolllery offers a better trade-off by avoiding the high costs of retraining and the rigidity of static weights.

The cost-benefit profile of Toolllery is uniquely optimized for real-world deployment. Although the offline phase utilizes high-capacity teacher LLMs to synthesize tool manuals, these costs are amortized over the lifetime of the tool library. The true advantage is realized during online inference. Without Toolllery, expanding the candidate pool from 10 to 1,000 tools results in a 35-fold increase in input tokens, pushing context lengths to $\sim 80\text{k}$ – 100k . This leads to a “performance cliff” where accuracy drops below 60% and computational latency increases significantly. Toolllery fixes the input token count at a nearly constant level by providing a compact, high-confidence candidate set. This efficiency enables smaller, low-latency models (e.g., Qwen3-4B) to operate in tool environments that were previously inaccessible, while allowing powerful LLMs to maintain peak accuracy

with drastically reduced inference costs.

Several directions remain promising for future work. First, we envision automating Tool User Manual maintenance by integrating Toolllery into RAG frameworks, ensuring real-time synchronization with evolving libraries. Second, replacing general-purpose embeddings with domain-adaptive encoders trained specifically for tool selection could further enhance precision. Finally, hybridizing Toolllery with lightweight adapters (e.g., LoRA) offers a promising middle ground, balancing the flexibility of retrieval with the specialized alignment of learned parameters.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, and 1 others. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Emre Can Acikgoz, Jeremiah Greer, Akul Datta, Ze Yang, William Zeng, Oussama Elachqar, Emmanouil Koukoumidis, Dilek Hakkani-Tur, and Gokhan Tur. 2025. Can a single model master both multi-turn conversations and tool use? coalM: A unified conversational agentic language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12370–12390.
- Mayank Agarwal, Ibrahim Abdelaziz, Kinjal Basu, Merve Unuvar, Luis A Lastras, Yara Rizk, and Pavan Kapanipathi. 2025. Toolrm: Outcome reward models for tool-calling large language models. *arXiv preprint arXiv:2509.11963*.
- Amazon Web Services. 2024. Boto3: The AWS SDK for Python. <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>. Accessed: 2024-05-01.
- Lutfi Eren Erdogan, Nicholas Lee, Siddharth Jha, Sehoon Kim, Ryan Tabrizi, Suhong Moon, Coleman Richard Charles Hooper, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. 2024. Tinyagent: Function calling at the edge. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 80–88.
- Tingchen Fu, Deng Cai, Lemao Liu, Shuming Shi, and Rui Yan. 2024. [Disperse-then-merge: Pushing the limits of instruction tuning via alignment tax reduction](#). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 2967–2985, Bangkok, Thailand. Association for Computational Linguistics.
- Shijue Huang, Wanjun Zhong, Jianqiao Lu, Qi Zhu, Jiahui Gao, Weiwen Liu, Yutai Hou, Xingshan Zeng, Yasheng Wang, Lifeng Shang, and 1 others. 2024. Planning, creation, usage: Benchmarking llms for comprehensive tool utilization in real-world complex scenarios. *arXiv preprint arXiv:2401.17167*.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, and 1 others. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526.
- Jianghao Lin, Xinyuan Wang, Xinyi Dai, Menghui Zhu, Bo Chen, Ruiming Tang, Yong Yu, and Weinan Zhang. 2025. Masstool: A multi-task search-based tool retrieval framework for large language models. *arXiv preprint arXiv:2507.00487*.
- Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, and 1 others. 2025. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*.
- Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, and 1 others. 2024a. Toolace: Winning the points of llm function calling. *arXiv preprint arXiv:2409.00920*.
- Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh RN, and 1 others. 2024b. Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37:54463–54482.
- Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang. 2025. An empirical study of catastrophic forgetting in large language models during continual fine-tuning. *IEEE Transactions on Audio, Speech and Language Processing*.
- Guozhao Mo, Wenliang Zhong, Jiawei Chen, Xuanang Chen, Yaojie Lu, Hongyu Lin, Ben He, Xianpei Han, and Le Sun. 2025. Livemcpbench: Can agents navigate an ocean of mcp tools? *arXiv preprint arXiv:2508.01780*.
- OpenAI. Introducing gpt-5. <https://openai.com/zh-Hans-CN/index/introducing-gpt-5/>. Accessed: 2025-12-31.
- Varatheepan Paramanayakam, Andreas Karatzas, Iraklis Anagnostopoulos, and Dimitrios Stamoulis. 2025. Less is more: Optimizing function calling for llm execution on edge devices. In *2025 Design, Automation & Test in Europe Conference (DATE)*, pages 1–7. IEEE.
- Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. 2025. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can

761 teach themselves to use tools. *Advances in Neural*
762 *Information Processing Systems*, 36:68539–68551.

763 Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier
764 Martinet, Marie-Anne Lachaux, Timothée Lacroix,
765 Baptiste Rozière, Naman Goyal, Eric Hambro,
766 Faisal Azhar, and 1 others. 2023. Llama: Open
767 and efficient foundation language models. *arXiv*
768 *preprint arXiv:2302.13971*.

769 Jun Wang, Jiamu Zhou, Xihuai Wang, Xiaoyun Mo,
770 Haoyu Zhang, Qiqiang Lin, Jincheng Jincheng,
771 Muning Wen, Weinan Zhang, and Qiuying Peng.
772 2025. Hammerbench: Fine-grained function-calling
773 evaluation in real mobile assistant scenarios. In
774 *Findings of the Association for Computational Lin-*
775 *guistics: ACL 2025*, pages 3350–3376.

776 Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin
777 Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and
778 Lingpeng Kong. 2024. Os-copilot: Towards gener-
779 alist computer agents with self-improvement. *arXiv*
780 *preprint arXiv:2402.07456*.

781 Hanwen Xu, Xuyao Huang, Yuzhe Liu, Kai Yu, and
782 Zhijie Deng. 2025. Tps-bench: Evaluating ai agents’
783 tool planning\& scheduling abilities in compound-
784 ing tasks. *arXiv preprint arXiv:2511.01527*.

785 An Yang, Anfeng Li, Baosong Yang, Beichen Zhang,
786 Binyuan Hui, Bo Zheng, Bowen Yu, Chang
787 Gao, Chengen Huang, Chenxu Lv, and 1 others.
788 2025. Qwen3 technical report. *arXiv preprint*
789 *arXiv:2505.09388*.

790 Shunyu Yao, Howard Chen, John Yang, and Karthik
791 Narasimhan. 2022. Webshop: Towards scalable
792 real-world web interaction with grounded language
793 agents. *Advances in Neural Information Processing*
794 *Systems*, 35:20744–20757.

795 Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin
796 Chen, Chengxing Xie, Cunxiang Wang, Da Yin,
797 Hao Zeng, Jiajie Zhang, and 1 others. 2025. Glm-
798 4.5: Agentic, reasoning, and coding (arc) foundation
799 models. *arXiv preprint arXiv:2508.06471*.